PhotoMatch — Manual Técnico de Defensa

Trabajo Práctico 09: DevSecOps y Observabilidad en Tiempo Real

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Seguridad: Dependabot + CodeQL + Secret Scanning

Release Oficial: `v9.0.0`
APM / Errores: Sentry (Node.js & Browser)
Monitoreo Externo: UptimeRobot (Incidente 503/UP)

A. Introducción Conceptual

1. Paradigma Shift-Left Security

Desplazar la seguridad a las etapas iniciales del ciclo de vida del software (en lugar de verificarla al final en producción) reduce drásticamente el costo de mitigación y previene brechas de seguridad:

- **Secret Scanning & Push Protection:** Intercepta secretos y tokens en el instante del commit.
- **Dependabot (SCA):** Monitorea dependencias y propone parches automatizados mediante Pull Requests.
- **CodeQL (SAST):** Analiza semánticamente el flujo de datos para detectar ReDoS e inyecciones.

2. Observabilidad: De la Detección a la Acción

- **Sentry (Caja Blanca):** Captura excepciones en tiempo real, stack traces y tiempos de transacción tanto en Express como en el navegador web.
- **UptimeRobot (Caja Negra):** Sondea desde Internet la salud de Producción, alertando ante degradaciones o caídas (HTTP 503).
- **Cierre de Ciclo:** Toda anomalía observada se traduce en una tarjeta en el tablero de trabajo de GitHub Projects.

B. Estado Inicial de PhotoMatch antes del TP9

- 4 dependencias transitivas vulnerables en el backend.
- Sin análisis estático SAST ni escaneo automático de vulnerabilidades.
- Expresión regular de email vulnerable a ataques ReDoS por retroceso catastrófico.
- Endpoint `/health` expuesto a agotamiento de conexiones por denegación de servicio.
- Sin telemetría de errores ni monitoreo externo.

C. Implementación Paso a Paso

1. **Automatización de Seguridad:** Configuración de `.github/dependabot.yml` y activación de CodeQL default setup en GitHub Actions. `npm audit fix` eliminó las 4 vulnerabilidades transitivas.
2. **Remediaciones de Código por Alertas CodeQL:**
   - **ReDoS:** Validación de email refactorizada a algoritmo lineal seguro en `validationService.js`.
   - **DoS en `/health`:** Inclusión de rate limiting (máximo 60 peticiones por minuto con `express-rate-limit`).
3. **Integración de Sentry:** `@sentry/node` en Express (`instrument.js`) y `@sentry/browser` compilado con `esbuild` en frontend.

4. **Captura de Incidente Real y Gestión en Tablero:** Desconexión intencional de PostgreSQL generó la excepción fatal real **PHOTOMATCH-BACKEND-1**. Se creó la **Issue 38** vinculada al tablero de GitHub Projects en `Todo`.
5. **Monitoreo UptimeRobot:** Sondeo a `/health` de Producción registró el incidente real 503 (372997347011072787) y su posterior recuperación a `Up` tras 5m 04s.

D. Decisiones de Ingeniería y Trade-offs

Decisión Tomada	Alternativa Descartada	Justificación Técnica y Trade-off
Dependabot Automatizado	Revisiones manuales de CVEs	Crea Pull Requests de remediación continua evaluados por la suite de CI sin intervención humana previa.
CodeQL SAST en CI	Linters estáticos superficiales	CodeQL analiza flujo de datos semántico (taint tracking), detectando ReDoS que un linter estándar no ve.
Pausar UptimeRobot post-test	Dejar sondeo 24/7 en máquina local	Evita falsos positivos de caída cuando la laptop del desarrollador entra en suspensión o se apaga.

E. Pruebas y Evidencias Reales

Evidencias Auditables en GitHub y Consolas

1. **PR de Dependabot Integrado:** PR 30 (`pg` 8.23.0 -> 8.23.1) fusionado exitosamente en commit `43f142e` con CI verde.
2. **CodeQL y Auditoría:** análisis activo, pero alerta HIGH 2 de rate limiting en login abierta en `main`; PR 41 corrige y espera merge. `npm audit` reportó 0 vulnerabilidades de dependencias.
3. **Sentry:** incidente fatal PHOTOMATCH-BACKEND-1 con entorno `qa`. Explore muestra trazas backend `prod`: GET /clients fue la más usada (6); POST /login, la más lenta por p75 (2,23 s). Frontend tiene pageloads, pero el entorno 127.0.0.1 aún mezcla QA/PROD; PR 42 lo corrige.
4. **GitHub Projects:** Issue 38 vinculada al tablero oficial en columna `Todo`.
5. **UptimeRobot:** Incidente real resuelto 372997347011072787 (DOWN 503 detectado, notificación y recuperación tras 5m 04s).
6. **Release Oficial:** `v9.0.0` publicada en GitHub.

F. Preparación para la Defensa Oral

Pregunta 1: ¿Qué es Shift-Left Security y cómo se evidencia en PhotoMatch?

Respuesta para defender:

Es la práctica de incorporar la seguridad en fases tempranas. En PhotoMatch se evidencia en tres barreras: Push Protection al hacer git push, Dependabot analizando dependencias en el repositorio, y CodeQL evaluando el código en cada PR antes de mergear.

Pregunta 2: ¿Cómo solucionaron la vulnerabilidad ReDoS detectada por CodeQL?

Respuesta para defender:

Reemplazamos la expresión regular vulnerable por verificaciones lineales de subcadenas (`indexOf`, `slice`) con complejidad $O(N)$ acotada por una guarda inicial de 254 caracteres, garantizando que el event loop de Node.js nunca se bloquee.

Pregunta 3 (Difícil): ¿Por qué agregaron un rate limiter al endpoint `/health` si suele ser público?

Respuesta para defender:

Porque nuestro healthcheck es «profundo»: ejecuta `SELECT 1` sobre PostgreSQL. Sin rate limiting, un atacante podría inundar `/health` para agotar las conexiones del pool de la base de datos y provocar una denegación de servicio a los usuarios reales.

G. Resumen Final de Estudio

- **Shift-Left:** Dependabot, CodeQL, Push Protection y saneamiento de dependencias.
- **Observabilidad:** Sentry (APM / Errores reales) y UptimeRobot (Disponibilidad externa).
- **Cierre de Ciclo:** Incidente de Sentry traducido a la tarjeta 38 en GitHub Projects.
- **Pitch de 2 minutos:** *«TP9 integra Dependabot (30), CodeQL y push protection. CodeQL analiza correctamente, pero hay una alerta HIGH abierta en login; PR 41 propone limitar intentos. Sentry capturó un error QA que originó la tarjeta 38 y mostró una tabla de trazas reales PROD; los pageloads de browser aún mezclan entornos hasta PR 42 UptimeRobot registró un DOWN 503 real y el UP tras 5 minutos; el monitor quedó pausado.»*